

MODÜL 01

Sinir Ağları Temelleri

Neural Networks Fundamentals

Biyolojik nörondan sıfırdan MLP'ye · NumPy · TensorFlow · PyTorch

Seviye	Başlangıç — Sıfırdan İleri Seviye
Framework	NumPy · TensorFlow/Keras · PyTorch
Bölümler	9 Ana Bölüm + Alıştırmalar + Kaynaklar
Dosyalar	.py (tam implementasyon) + .ipynb (kapsamlı notebook)
Sonraki Modül	Modül 02 — Aktivasyon Fonksiyonları (Derinlemesine)

Modül Konsept Haritası

Biyolojik Nöron	→	Yapay Nöron	→	Perceptron
Dendrit/Sinaps		$x \cdot w + b$		Doğrusal Sınıf.
		↓		↓
		Aktivasyon $f(z)$		XOR Problemi
		σ , tanh, ReLU		Tek katman yetersiz
		↓		↓
İleri Besleme	←	MLP Mimarisi	→	Backpropagation
Forward Pass		Çok Katmanlı		Zincir Kuralı

1. Öğrenme Hedefleri

- ✓ Yapay bir nöronun matematiksel modelini formüle etmek: $z = w \cdot x + b$, $a = f(z)$
- ✓ İleri besleme (forward pass) işlemini matris çarpımı olarak hesaplamak
- ✓ XOR problemini örnek alarak neden gizli katmanın zorunlu olduğunu kanıtlamak
- ✓ Binary Cross-Entropy kayıp fonksiyonunu türetmek ve yorumlamak
- ✓ Geri yayılım algoritmasını zincir kuralıyla adım adım uygulamak
- ✓ Sigmoid, Tanh ve ReLU aktivasyonlarının farkını grafiksel olarak göstermek
- ✓ Aynı ağı NumPy (from scratch), TensorFlow/Keras ve PyTorch ile kodlamak
- ✓ Öğrenme hızı ve epoch sayısının eğitime etkisini deneysel olarak ölçmek

2. Teorik Temel

2.1 Biyolojik Nöron → Yapay Nöron

Beyin yaklaşık 86 milyar nörondan oluşur. Her nöron dendritler aracılığıyla sinyal alır, bu sinyalleri hücre gövdesinde toplar ve belirli bir eşiği aşarsa aksonu üzerinden çıkış üretir. McCulloch ve Pitts (1943), bu biyolojik süreci matematiksel olarak modellemiştir. Rosenblatt (1958) ise bu modelden öğrenen ilk algoritmayı — Perceptron'u — geliştirmiştir.

Biyolojik Nöron	Matematiksel Karşılık	Yapay Sinir Ağı Bileşeni
Dendrit	Giriş sinyalleri	$x_1, x_2, \dots, x_n$
Sinaptik ağırlık	Bağlantı gücü	$w_1, w_2, \dots, w_n$ (öğrenilir)
Hücre gövdesi	Toplama işlemi	$z = w \cdot x + b$
Akson hillock	Ateşlenme eşiği	Aktivasyon fonksiyonu $f(z)$
Akson / Çıkış	İletilen sinyal	$a = f(z)$

2.2 Tek Nöron Matematiği

Bir yapay nöron iki adımda çalışır:

Adım 1 — Ağırlıklı Toplam (Weighted Sum):

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b = w^T \cdot x + b$$

Burada x_i giriş özellikleri, w_i öğrenilen ağırlıklar ve b bias terimidir. Bias, nöronun eşik değerini ayarlar — bu olmazsa nöron her zaman orijinden geçen bir hiper-düzlem öğrenir ve öğrenme kapasitesi kısıtlanır.

Adım 2 — Aktivasyon Fonksiyonu:

$$a = f(z)$$

Aktivasyon fonksiyonu f , ağı doğrusal olmayan (non-linear) kapasite kazandırır. Aktivasyonsuz bir ağı, ne kadar derin olursa olsun yalnızca doğrusal dönüşüm yapabilir ve

karmaşık örüntüleri öğrenemez.

2.3 Sayısal Örnek — Adım Adım

Değişken	Değer	Açıklama
$x = [x_1, x_2, x_3]$	[0.5, -1.2, 0.8]	Giriş özellikleri
$w = [w_1, w_2, w_3]$	[0.4, -0.3, 0.7]	Ağırlıklar (öğrenilmiş)
b	0.1	Bias terimi
$z = w \cdot x + b$	$0.4 \times 0.5 + (-0.3) \times (-1.2) + 0.7 \times 0.8 + 0.1$	Ağırlıklı toplam
	$= 0.20 + 0.36 + 0.56 + 0.10 = 1.22$	
$a = \sigma(z)$	$1 / (1 + e^{(-1.22)}) = 0.7721$	Sigmoid aktivasyon
Yorum	Bu nöron %77.21 olasılıkla "ateşleniyor"	İkili sınıflandırma

3. Görsel Şemalar

Şema 1: Tek Nöron Mimarisi

GİRİŞLER	→	AĞIRLIKLAR	→	NÖRON	→	ÇIKIŞ
$x_1 = 0.5$		$w_1 = 0.4$				
$x_2 = -1.2$		$w_2 = -0.3$		$z = w \cdot x + b = 1.22$		$a = \sigma(z)$
$x_3 = 0.8$		$w_3 = 0.7$		$b = 0.1$		$= 0.7721$

Şekil 1: Tek yapay nöronun giriş → ağırlık → toplama → aktivasyon adımları.

Şema 2: MLP (Çok Katmanlı Perceptron) Mimarisi

XOR problemini çözmek için 2 giriş → 4 gizli nöron → 1 çıkış mimarisi:

GİRİŞ KATMANI		GİZLİ KATMAN		ÇIKIŞ KATMANI
x_1 (bit 1)		h_1		
		h_2		$\hat{y}$ (XOR tahmini)
x_2 (bit 2)		h_3		
		h_4		
2 nöron	$W_1(2 \times 4)$	4 nöron	$W_2(4 \times 1)$	1 nöron
	$b_1(1 \times 4)$	Sigmoid	$b_2(1 \times 1)$	Sigmoid

Şekil 2: XOR için MLP mimarisi — parametre boyutları.

Şema 3: Forward Pass Hesaplama Adımları

Adım 1: $Z_1 = X @ W_1 + b_1$ boyutlar: $(4,2) @ (2,4) + (1,4) = (4,4)$

Adım 2: $A_1 = \text{sigmoid}(Z_1)$ boyutlar: $(4,4)$

Adım 3: $Z_2 = A_1 @ W_2 + b_2$ boyutlar: $(4,4) @ (4,1) + (1,1) = (4,1)$

Adım 4: $A_2 = \text{sigmoid}(Z_2) = \hat{y}$ boyutlar: $(4,1) \leftarrow$ XOR tahmini

4. Aktivasyon Fonksiyonları

Aktivasyon fonksiyonları ağı doğrusal olmayan kapasite kazandıran kritik bileşenlerdir. Aktivasyonsuz bir ağ ne kadar derin olursa olsun sadece doğrusal dönüşüm yapabilir — XOR gibi problemleri çözemez.

Fonksiyon	Formül	Aralık	Avantajı	Dezavantajı
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$	(0, 1)	Olasılık çıktısı, pürüzsüz	Vanishing gradient, yavaş
Tanh	$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$	(-1, 1)	Sıfır merkezli, sigmoid'den hızlı	Hâlâ vanishing gradient
ReLU	$\max(0, z)$	$[0, +\infty)$	Hızlı, basit, vanishing yok	Dead neuron ($z < 0$)
Leaky ReLU	$\max(0.01z, z)$	$(-\infty, +\infty)$	Dead neuron yok, hızlı	Negatif bölge sabitini seçmek
GELU	$z \cdot \Phi(z)$	$(\approx -0.17, +\infty)$	BERT/GPT'de kullanılır, SOTA	hesaplaması daha pahalı

4.1 Vanishing Gradient Problemi

Sigmoid'in türevi: $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$. z büyük veya küçük olduğunda bu türev 0'a yaklaşır. Derin ağlarda gradyanlar katman katman geriye iletilirken üst üste çarpılır ve sıfıra yaklaşır. ReLU bu problemi büyük ölçüde çözer: $z > 0$ için türev her zaman 1'dir.

z değeri	$\sigma(z)$	$\sigma'(z) = a \cdot (1-a)$	Gradient durumu
-10	0.0000454	≈ 0.0000454	⚠ Çok küçük — vanishing!
-5	0.00669	0.00665	Küçük
0	0.500	0.250	✓ Maksimum değer: 0.25
5	0.99331	0.00665	Küçük
10	0.9999546	≈ 0.0000454	⚠ Çok küçük — vanishing!

5. XOR Problemi ve Gizli Katmanın Gerekliliği

XOR (Exclusive OR) problemi, derin öğrenme tarihinde kritik bir rol oynamıştır. 1969'da Minsky ve Papert, tek katmanlı Perceptron'un XOR'u çözemeyeceğini matematiksel olarak kanıtladı. Bu 'AI Winter'ın başlangıcıydı. Gizli katman eklenmesiyle problem çözülebilir — bu keşif modern derin öğrenmenin temelidir.

5.1 XOR Doğruluk Tablosu

x_1	x_2	XOR Çıktısı	Açıklama
0	0	0	Her ikisi de 0 → çıkış 0
0	1	1	Biri 1, diğeri 0 → çıkış 1
1	0	1	Biri 1, diğeri 0 → çıkış 1
1	1	0	Her ikisi de 1 → çıkış 0 (XOR paradoksu)

Kanıt: (0,0) ve (1,1) → sınıf A (XOR=0), (0,1) ve (1,0) → sınıf B (XOR=1). Bu dört nokta 2D uzayda hiçbir tek doğruyla ayrılamaz — lineer ayrılmazlığın klasik örneği. Gizli katman bu veriyi yeni bir uzaya taşır ve lineer ayrılabilir hale getirir.

5.2 Neden Gizli Katman Çözer?

- Gizli katman nöronları $h_1 \approx (x_1 \text{ NAND } x_2)$, $h_2 \approx (x_1 \text{ OR } x_2)$ işlemlerini öğrenir
- Yeni özellik uzayı: $(h_1, h_2) = \{(1,0), (1,1), (1,1), (0,1)\}$ olur
- Bu uzayda (1,1) → XOR=1 ve diğerleri → XOR=0, tek çizgiyle ayrılabilir
- Evrensel Yaklaşım Teoremi: En az 1 gizli katmanlı MLP, yeterli nöronla her sürekli fonksiyonu yaklaşık öğrenebilir

6. Geri Yayılım (Backpropagation)

Backpropagation, kayıp fonksiyonunun ağırlıklara göre türevini hesaplamak için zincir kuralını (chain rule) sistematik şekilde uygular. 1986'da Rumelhart, Hinton ve Williams tarafından popülerleştirilen bu algoritma, derin öğrenmenin temel eğitim motorudur.

6.1 Zincir Kuralı

$$dL/dW_1 = dL/dA_2 \cdot dA_2/dZ_2 \cdot dZ_2/dA_1 \cdot dA_1/dZ_1 \cdot dZ_1/dW_1$$

6.2 Çıkış Katmanı Gradyanları

1. $dL/dA_2 = (A_2 - Y) / n$ [BCE + Sigmoid türevi – güzel basitleşme]
2. $dW_2 = A_1^T @ (dL/dA_2)$ [boyut: (4,1)]
3. $db_2 = \Sigma(dL/dA_2, \text{axis}=0)$ [boyut: (1,1)]

6.3 Gizli Katman Gradyanları

4. $dL/dA_1 = (dL/dA_2) @ W_2^T$ [boyut: (n,4)]
5. $\delta_1 = dL/dA_1 \cdot A_1 \cdot (1-A_1)$ [sigmoid türevi – element-wise çarpım]
6. $dW_1 = X^T @ \delta_1$ [boyut: (2,4)]
7. $db_1 = \Sigma(\delta_1, \text{axis}=0)$ [boyut: (1,4)]

6.4 Ağırlık Güncellemesi

$$W \leftarrow W - \eta \cdot dW \quad (\eta = \text{öğrenme hızı, negatif yönde adım})$$

Negatif yönde adım atıyoruz çünkü kaybı minimize etmek istiyoruz. Gradyan fonksiyonun en hızlı arttığı yönü gösterir; biz tam tersine gidiyoruz.

7. Framework Karşılaştırması

Özellik	NumPy (Scratch)	TensorFlow/Keras	PyTorch
Kodu kim yazar	Biz (her şey)	Keras API	nn.Module
Autograd	Hayır	Evet	Evet
GPU desteği	Hayır	Evet	Evet
Esneklik	En yüksek	Orta	Çok yüksek
Üretim (Prod.)	Hayır	Evet	Evet (TorchServe)
Araştırma	Eğitim amaçlı	Orta yaygınlık	En yaygın
Öğrenme eğrisi	En düşük	Orta	Orta
Ne zaman kullan	Matematiği öğrenmek	Hızlı prototip	Özel mimari/araştırma

8. Yaygın Hatalar ve Tuzaklar

Hata 1: Ağırlıkları sıfır ile başlatmak	Tüm nöronlar aynı gradyanı alır, simetri kırılmaz ve ağ hiçbir şey öğrenemez. Çözüm: Küçük rastgele değerler veya Xavier/He başlatma kullanın.
Hata 2: Çok büyük öğrenme hızı seçmek	Loss salınım yapar ve ıraksayabilir (diverge). Çözüm: $lr=0.001$ ile başlayın, gerekirse artırın. Learning rate finder kullanın.
Hata 3: Aktivasyon fonksiyonunu unutmak	Aktivasyonsuz ağ ne kadar derin olursa olsun yalnızca doğrusal dönüşüm yapar — XOR gibi problemleri çözemez. Çözüm: Her gizli katmana mutlaka aktivasyon ekleyin.
Hata 4: PyTorch'ta gradyanları sıfırlamamak	PyTorch gradyanları her backward() çağrısında biriktirir. Çözüm: Her epoch başında mutlaka optimizer.zero_grad() çağırın.
Hata 5: Eğitim verisiyle test yapmak	Model ezberlediği veriyi test edince yanıltıcı yüksek performans görürsünüz. Çözüm: Her zaman ayrı test seti kullanın; validation loss'u takip edin.
Hata 6: Loss azalırken accuracy'e bakmamak	Loss azalabilir ama eşik değeri (threshold=0.5) yanlış seçilmişse accuracy düşük kalır. Çözüm: Her iki metriği birlikte takip edin; ROC eğrisini inceleyin.

9. Özet Tablo

Kavram	Formül / Kural	Ne Zaman Kullanılır
Ağırlıklı toplam	$z = w^T x + b$	Her katman forward pass
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$	Çıkış katmanı (ikili sınıf.)
Tanh	$\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$	Gizli katman (sıfır merkezli)
ReLU	$\max(0, z)$	Modern gizli katmanlar
BCE Kaybı	$-1/n \cdot \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	İkili sınıflandırma
MSE Kaybı	$1/n \cdot \sum (y - \hat{y})^2$	Regresyon problemleri
Gradient Descent	$W \leftarrow W - \eta \cdot \partial L / \partial W$	Her epoch ağırlık güncelleme
Xavier Başlatma	$W \sim N(0, 2/(n_{in} + n_{out}))$	Ağırlık başlangıç değeri
Forward Pass	$X \rightarrow Z_1 \rightarrow A_1 \rightarrow Z_2 \rightarrow A_2 = \hat{y}$	Tahmin üretme
Backpropagation	Zincir kuralı: $\partial L / \partial W$	Eğitim (gradient hesabı)

10. Kaynaklar ve İleri Okuma

- [1] McCulloch, W.S. & Pitts, W. (1943). A Logical Calculus of Ideas Immanent in Nervous Activity. Bulletin of Mathematical Biophysics.
- [2] Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. Psychological Review.
- [3] Rumelhart, D.E., Hinton, G.E. & Williams, R.J. (1986). Learning Representations by Back-Propagating Errors. Nature, 323, 533-536.
- [4] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press. — Bölüm 6: Deep Feedforward Networks.
- [5] Nielsen, M. (2015). Neural Networks and Deep Learning. <http://neuralnetworksanddeeplearning.com> — Ücretsiz online kitap.
- [6] CS231n Stanford — Convolutional Neural Networks for Visual Recognition. <http://cs231n.stanford.edu>
- [7] PyTorch Resmi Dokümantasyon — nn.Module ve Autograd. <https://pytorch.org/docs>

Sonraki Adım: Modül 02 — Aktivasyon Fonksiyonları | Vanishing Gradient • ELU • SELU • Swish • GELU • Karar Rehberi